

Assignment 3: Architecture Diagram

System Architecture Diagram

The diagram below shows the complete flow of a single user request through the Personal Productivity and Task Execution Agent — from the moment it is typed in the Frontend, through Agent State / reasoning, tool selection, the human-approval gate, tool execution against the database, error handling, and finally execution logging for observability.

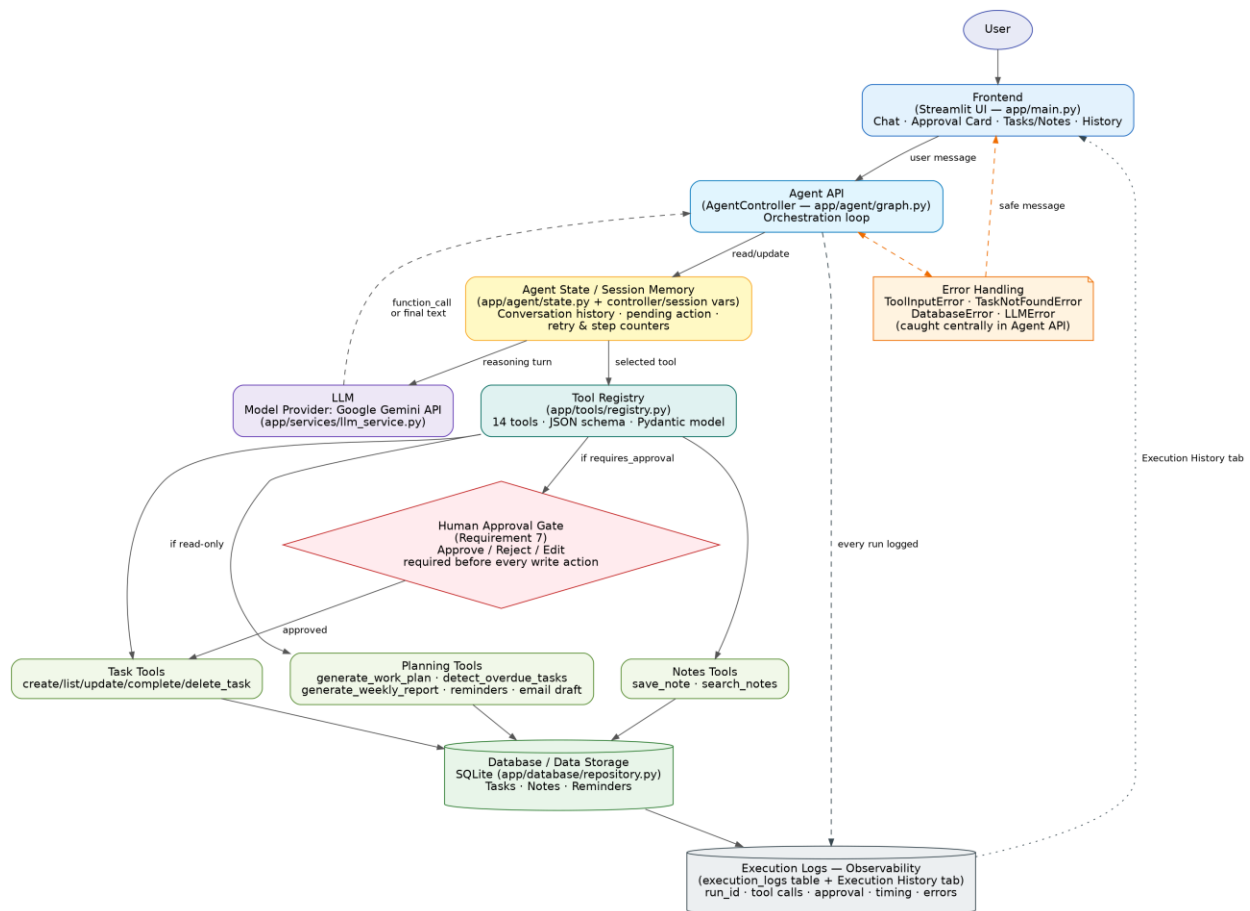


Figure 1: End-to-end system architecture

Legend: solid arrows = the main request path (Frontend → Agent API → Agent State → LLM/Tool Registry → tools → Database → Execution Logs). The red diamond is the Human Approval Gate, sitting between tool selection and execution for any write action. The orange note is Error Handling, which the Agent API consults on every step and which always reports back to the Frontend as a safe, human-readable message rather than a raw exception.

Component-by-Component Explanation

1. Overview of the Architecture

The system follows a strict layered pipeline rather than a single monolithic chatbot loop: User → Frontend → Agent API → Agent State → LLM / Tool Registry → Human Approval Gate (write

actions only) → Tool Execution → Database → Execution Log → back to the Frontend as a final response. Every layer has a single responsibility, which is what makes the agent's decisions — which tool, whether approval is needed, when to stop — auditable rather than opaque.

2. User

The human operator of the system — a student or professional organizing tasks, notes, and daily/weekly plans. The user only ever interacts with the Frontend layer; they never call tools or the LLM directly, which keeps the system safe and predictable regardless of what the user types.

3. Frontend (Streamlit UI — `app/main.py`)

The presentation layer. It renders the chat input and conversation history, a live Agent Status panel (Thinking, Selecting a tool, Waiting for approval, Executing a tool, Producing a final response, Handling an error), the human-approval card with Approve / Reject / Edit-arguments controls, a Tasks/Notes panel backed directly by the database, and an Execution History tab for reviewing past runs. The Frontend contains no business logic — it only renders state produced by the Agent API and forwards user input back to it.

4. Agent API (AgentController — `app/agent/graph.py`)

The orchestrator and the only component that talks to the LLM. On every user turn it: sends the request (and full history) to Gemini, interprets whether a tool call or a direct answer came back, checks whether the selected tool requires approval, executes the tool (with a worker-thread timeout and retry/back-off), validates the result, and produces the final natural-language response. It also enforces the execution limits from Requirement 9 — maximum agent steps, duplicate tool-call detection, per-tool retry caps — so the agent can never run away or repeat an action indefinitely.

5. Agent State and Session Memory

Holds everything the agent needs to remember for the current session: the running Gemini conversation history (so references like "mark the second one complete" resolve correctly without custom entity-tracking code), the currently pending approval (tool name, arguments, expected effect), and the step/retry counters used for the execution limits. In this project, session memory is split across two concrete places — Streamlit's `st.session_state` (message list, pending approval, UI status) and the AgentController instance itself (chat session, step/retry counters) — both of which live only for the duration of the browser session; anything that must outlive the session (tasks, notes, reminders, execution logs) is written to the Database instead.

6. LLM and Model Provider

The reasoning engine. This project uses Google's Gemini API as the model provider (`app/services/llm_service.py`), given a system prompt (Requirement 12), the current date, and the JSON-schema function declarations of all 14 available tools. On each turn the LLM returns either a function call (naming a tool and its arguments) or a final plain-language answer. The Agent API never exposes the model's private step-by-step reasoning to the user — only short, human-readable operational statuses (e.g. "Selecting a tool...") are shown.

7. Human Approval Gate

A mandatory checkpoint, enforced in code inside the Agent API — not left to the LLM's discretion alone — before any write or irreversible action executes: updating a task, completing a task, deleting a task, creating a reminder, drafting a follow-up email, or creating more than one task in a single request. The Frontend displays the proposed tool name, its input arguments (editable before approving), and the expected effect, with Approve / Reject / Edit controls. A rejected action is never executed; control simply returns to the user. Because the gate lives in the Agent API rather than in a prompt instruction, it cannot be silently skipped even if the model's phrasing changes.

8. Tool Registry (`app/tools/registry.py`)

The single source of truth that maps every tool name to its Pydantic input model, its handler function, and its approval flag, and converts that information into the JSON function-declaration schemas the LLM uses to decide which tool to call. Because every tool is registered in one place, adding a new capability never requires touching the Agent API itself — only a new tool file plus one registry line.

9. Task Tools

Implements `create_task`, `list_tasks`, `update_task`, `complete_task`, and `delete_task` — full task-lifecycle management, from creation (priority/due-date/tags) through filtered listing, field updates, completion, and deletion. `update_task`, `complete_task`, and `delete_task` are all gated behind human approval as write actions; `list_tasks` is read-only and needs none.

10. Notes Tools

Implements `save_note` and `search_notes`. Notes are searchable both by exact keyword match and by a lightweight fuzzy-matching score (Python's `difflib.SequenceMatcher`), giving forgiving, typo-tolerant relevance ranking even without a full external vector database.

11. Planning Tools

Implements `generate_work_plan` together with the bonus tools `create_reminder`, `list_reminders`, `detect_overdue_tasks`, `generate_weekly_report`, and `draft_follow_up_email`. `generate_work_plan` reads the current task list from the Database and applies deterministic scoring logic — priority, deadline proximity, and estimated effort — so the same input always produces the same, explainable daily schedule, with no LLM call involved in the scheduling itself.

12. Database and Data Storage

A SQLite database provides durable, restart-safe persistence for tasks, notes, reminders, and execution logs. Storage code is fully isolated behind a repository layer (`app/database/repository.py`), so the agent and tools never touch SQL directly — swapping SQLite for PostgreSQL or Supabase later would only touch this one file.

13. Error Handling

Every failure mode required by the specification is handled explicitly, each as its own exception type: missing API key, invalid tool arguments (`ToolInputError`), unknown task/note ID (`TaskNotFoundError` / `NoteNotFoundError`), database failure (`DatabaseError`), empty user input, tool execution timeout, and LLM/Gemini API errors including quota or rate-limit errors (`LLMError`, retried with back-off). The Agent API catches all of these centrally, so in every case

the user sees a short, plain-language message in the chat — never a raw stack trace, and never a secret such as the API key.

14. Execution Logs and Observability

Every agent run is recorded with a run ID, the original user request, the selected model, every tool called with its arguments and result, the approval status of each action, any errors, start/end timestamps, total duration, and the final outcome — but never API keys or private chain-of-thought reasoning. The Frontend's Execution History tab is the observability interface for reviewing any past run, which is essential both for debugging and for demonstrating, in a live code review, exactly how the agent behaved on a given request.